Modelo de datos NoSQL / especializado

TimescaleDB + PostGIS para predicción de demanda

García, Barreto, Chulluncuy, Hilario

Base de Datos (MCD8008) – Parte 2

1. Principios de diseño

El modelo especializado **no reemplaza** al modelo relacional. Se trata de una capa analítica que reutiliza las dimensiones (`core.zona_taxi`, `core.plataforma`) y proyecta la serie de demanda en una estructura optimizada para consultas temporales y espaciales.

- **Polyglot Persistence:** PostgreSQL conserva el núcleo transaccional; la capa TimescaleDB/PostGIS se deriva de él y se reconstruye desde el agregado `analytics.demanda_hora`.
- **Desnormalización deliberada y reconstruible:** el hypertable duplica la demanda horaria para acelerar el acceso, igual que ya hacía `demanda_hora` en la Parte 1.
- **Separación temporal–espacial:** las coordenadas viven en la entidad maestra `zona_taxi` (normalizada) y se unen en tiempo de consulta; así el hypertable permanece compresible (las columnas geográficas no son compresibles de forma nativa).

2. Capa espacial (PostGIS)

2.1. Atributo geo en la entidad zona_taxi

```
ALTER TABLE core.zona_taxi
    ADD COLUMN IF NOT EXISTS geo GEOGRAPHY(POINT, 4326);

UPDATE core.zona_taxi
SET geo = ST_SetSRID(ST_MakePoint(longitud, latitud), 4326)::geography
WHERE longitud IS NOT NULL AND latitud IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_zona_taxi_geo
    ON core.zona_taxi USING GIST (geo);
```

Se usa el tipo `GEOGRAPHY(POINT,4326)` (WGS84) para que las funciones de distancia (`ST_Distance`, `ST_DWithin`) devuelvan valores en metros sobre el elipsoide, adecuados para el dominio de NYC.

3. Hypertable de demanda (TimescaleDB)

3.1. Estructura

```
CREATE TABLE analytics.demanda_serie (
    hora TIMESTAMPTZ NOT NULL,
    zona_origen_id SMALLINT NOT NULL,
    plataforma_codigo VARCHAR(10) NOT NULL,
    viajes BIGINT NOT NULL
```

```
);

SELECT create_hypertable('analytics.demanda_serie', 'hora',
                         if_not_exists => TRUE);
```

La columna `hora` es la dimensión de particionamiento: TimescaleDB la divide en *chunks* de tamaño automático, de modo que una consulta por rango temporal solo accede a los chunks relevantes.

3.2. Población

```
INSERT INTO analytics.demanda_serie
    (hora, zona_origen_id, plataforma_codigo, viajes)
SELECT hora::timestamptz, zona_origen_id, plataforma_codigo, viajes
FROM analytics.demanda_hora;
```

4. Continuous aggregate

```
CREATE MATERIALIZED VIEW analytics.demanda_diaria
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 day', hora) AS dia,
       zona_origen_id,
       plataforma_codigo,
       SUM(viajes) AS viajes,
       COUNT(*) AS n_horas
FROM analytics.demanda_serie
GROUP BY dia, zona_origen_id, plataforma_codigo
WITH NO DATA;

CALL refresh_continuous_aggregate('analytics.demanda_diaria', NULL, NULL);
```

El agregado diario se mantiene de forma incremental y automática, evitando recalcular agregaciones completas en cada consulta.

5. Política de compresión

```
ALTER TABLE analytics.demanda_serie SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'zona_origen_id, plataforma_codigo',
    timescaledb.compress_orderby = 'hora'
);

SELECT add_compression_policy('analytics.demanda_serie', INTERVAL '7 days');
```

Los segmentos se comprimen por `zona_origen_id` y `plataforma_codigo`, y se ordenan por `hora`. Resultado observado: 162 MB (relacional) $\rightarrow$ 14 MB (comprimido), factor $\approx 11{,}6\times$.

6. Esquema resultante

Objeto	Tecnología / tipo
`core.zona_taxi.geo`	PostGIS `GEOGRAPHY(POINT,4326)` + índice GiST
`analytics.demanda_serie`	Hypertable (dimensión `hora`)
`analytics.demanda_diaria`	Continuous aggregate diario
Compresión	Segmento por (zona, plataforma), orden por hora